Is the quantitative code correct, and can it be simpler?

Independent review of the active model and its computational structure

Prepared for Tommaso De Santo

5 September 2026

5 September 2026. Original read-only review, followed by a focused repair.

The closing repair addendum records the subsequent saved-policy fix, bounded-price search cleanup and corrected reading map. Findings and source-line references in the original review below describe the frozen pre-repair code.

The code supports continued, carefully checked calibration work. I did not find a new defect that invalidates the selected calibration or its saved five-policy comparison. I did find a reproducible state-management defect that can corrupt a reused policy after another price evaluation, misleading fertility output, and a failure path that repeats an identical expensive calculation. These deserve targeted repairs before more elaborate policy experiments.

The code is substantially overgrown. The problem is not simply its size: old and current economic architectures share large mutable modules, a supposedly current entry point launches an obsolete model, and core distribution logic appears in several forms. The main speed opportunities are fewer complete household solutions and less repeated distribution work. Moving old files out of sight will make the project easier to understand, but will not by itself make calibration materially faster.

Recommended order: make saved policies self-contained; provide one reliable entry point and current model map; measure the time spent in the active Bellman and distribution operators; then simplify those operators in small, verified steps. Avoid a wholesale rewrite during the current calibration.

1. Scope, source identity, and limits

The authoritative specification for this review is the maintained September 4 production contract in CALIBRATION_STATUS.md S001. Its complete target-fit and parameter tables remain the reference; this review neither estimates nor promotes a calibration.

The reviewed household code is intergen_eqscale_seq_optimized/solver.py S002, its kernels, parameters, target measurement, and the dated calibration and policy drivers. The scientific runtime was checked against the frozen production snapshot at:

`/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/tmp/e5f_overnight_20260905a/frozen_production/code/model/`

The production bundle contains **50 files**, with SHA-256 `630ba20bca6a1b54eb4c46aca904c4a087afb8c808b9c7f4660d5fcd316a970e`. The working checkout's corresponding contract contains **51 files**, with SHA-256 `b033604b0b647f200bb03c5260fa476bcf61cccbb1689b04e76c745aa8f216a6`: it adds the default-off young-ownership profile and changes the calibration driver to expose that option. The other 49 production entries match. The core solver and calendar-policy defects below are present in the exact frozen production sources. The new morning orchestration is owned and reviewed separately by the lead.

This is a full architecture review and a targeted mathematical/correctness inspection of the active path, not a line-by-line certification of every historical experiment. No model solve, cluster job, refactoring, target change, or manuscript edit was performed. The existing PDFs provide the broader empirical and economic audit: full audit source S003, especially Sections 16-18.

The two supposedly current package guides are themselves historical: the package README still directs the reader to a June diagnostic, and `IMPLEMENTATION_STATUS.md` is dated June 25 and describes one-shot fertility and old defaults. Where they disagree with live status, their claims are not a new model contract. No substantive source change was made to reconcile them during this review.

Reading guide for a returning author

Read the current target and parameter tables in `CALIBRATION_STATUS.md` first. Then use the following seven stops; there is no need to start at line one of the 7,428-line solver.

1. **Experiment and configuration:** calibration main S004. Follow the profile, old-state normalization, age bridge, dated preference path and target callbacks. Generic parameter defaults are overridden here and in the selected profile; they are not the production estimate.
2. **Preferences and household needs:** precompute_shared S005. This translates parity and dependent children into housing floors, child utility and the equivalence scale.
3. **Decision order and expectations:** solve_bellman_full_markov_income S006. Read its backward age loop, tenure stage and final fertility stage. This is the active household problem.
4. **Conditional optimization and financing:** renter and owner kernels S007, with the tenure logit immediately above them. These implement saving, consumption, usable housing services and the purchase cash test.
5. **Where households go next:** advance_cohort_one_period_markov_income S008. It moves mass through location, tenure, saving, income and child maturation. Compare it with current-choice realization when distinguishing wealth from housing measurements.
6. **How the principal housing target is measured:** begin_dated_first_birth_housing_branch S009 and the following `finish` function. These explain the treated/control experiment more directly than the generic statistics code.
7. **Calendar population and policy changes:** run_birth_vintage_scenario S010, followed by apply_policy S011. The former separates inherited mass, births, entry and age raking; the latter shows exactly which economic primitives each policy changes.

2. Confirmed defects and their exposure

C1. Reused calendar policies can read continuation fertility from a different solve - high priority

Locations: PolicyBundle S012, solve_policy S013, evaluate_period S014, and sequential fertility S015.

A `PolicyBundle` is the object holding a solved household policy at a particular price. It contains first-birth, saving, housing and tenure choices, but **does not contain second- and third-birth probabilities**. The Bellman solver instead writes these probabilities to the mutable parameter object, `P._fert2_probs`. The forward fertility operator reads that field regardless of which policy bundle was supplied.

Consequently, evaluating price A, then price B, then reusing the stored policy from A can combine A's first-birth and tenure choices with B's continuation-birth probabilities. The stationary equilibrium code already solves this same problem by carrying and restoring continuation probabilities in its retained payload; see the existing stationary repair S016.

Reproduction: a deterministic zero-solve example puts one unit of inherited mass in the one-child state. The identical supplied policy, price and inherited distribution yields zero births with its original continuation probabilities, then one birth after only `P._fert2_probs` is replaced. The actual `evalu–`

`ate_period` and sequential fertility functions execute; dead-state preparation and tenure realization are mocked as identities to isolate the defect. This is an API defect, not a claim that production births differ by one unit.

A concrete trigger exists in the tighter-tolerance retry S017 and post-2023 evaluate_state retry S018: a failed search mutates the field at rejected prices, then the retry passes the previous stored policy again. If that first retry evaluation is accepted, no new Bellman solve restores the correct probabilities.

Observed exposure: none of the 55 dates in the saved September 4 five-policy packet records a grid-resolution fallback. During the fitted five-date history, the preference shifter changes at every date and the driver invalidates the stored policy; the retry therefore solves afresh. There is no demonstrated effect on task_010 or the current same-contract historical refinement. Exact repetition alone would not detect this deterministic defect if an affected path were exercised.

Minimal remedy: add continuation probabilities to the policy bundle, return them from a solved policy, and pass them explicitly to fertility measurement. A compatibility shim that restores them before every evaluation is an acceptable first step. Test A to B to reuse-A, a forced tight-search failure to retry, serialization/reload, and a cached policy after a preference or LTV change. The test must compare the full distribution and births, not just market residuals.

Evidence and executable probe are retained in the JSON receipt S019 and the executable reproduction S020.

C2. The documented current entry point launches the June model - medium priority

Locations: package README S021, runner settings S022, and runner dispatch S023.

The README tells a returning researcher to run `run_intergen_model.py` for the current model. That runner loads a June 23 source, the historical `candidate_replacement_young_old_roomgap_v1` target set, and a 16-age/60-wealth-node grid. It invokes the June mechanics-packet tool. It does not reconstruct the maintained 17-age/120-node dated sequential calibration.

Effect: a user following the instructions sees an older economic model and older results while expecting the current one. Existing production launchers use explicit drivers and are unaffected.

Minimal remedy: relabel the old runner and historical README sections, and provide one maintained `inspect selected solution` command that loads the selected contract, reports its identity, and opens the stable diagnostic packet. Keep reconstruction and plot refresh separate so inspecting a result does not inadvertently start a fresh model solve.

C3. Verbose fertility output uses the withdrawn factor of two - medium priority for interpretation

Locations: run_model_cp_dt output S024, fixed-price output S025, and authoritative extraction S026.

Several verbose lines still print `2 * mean_parity` as "TFR." In the active literal-parity model, parity is already the number of births; the top-coded completed-fertility statistic uses the declared weight for the 3+ bin. A population in which everyone has exactly one child prints "TFR=2.00" through the old log formula, while authoritative extraction correctly returns one.

Effect: wrong console interpretation; no evidence that the production target-fit CSV uses this wrong formula. The target extractor and dated measurement explicitly implement the maintained convention.

Minimal remedy: use one named measurement function for output as well as scoring, label completed fertility as such, and distinguish explicit three-child counts from the top-coded statistic. Preserve the age population used for each statistic. A scalar alias such as `mean_completed_fertility` should not silently stand for several different age and top-code conventions.

C4. Failure at a price boundary repeats the same solve - confirmed efficiency defect

Location: calendar market bracketing S027.

When excess demand keeps the same sign at `p_max`, the expansion loop repeatedly clips its next trial to that same bound. It neither stops when the bound stops moving nor caches that price. A mocked positive-excess example produces **19 evaluations at only two distinct prices**, including 18 at the identical upper bound, before reporting that no bracket exists.

The residual mock proves redundant execution, not its empirical frequency in accepted runs. Each real evaluation on this path would invoke a full household solve. Successful production paths need not be affected.

Minimal remedy: stop expansion when the next price equals the current bound, or reuse an exact-price cache. Keep failure explicit. The final exhausted bisection also recomputes its last midpoint once; this repeat can be removed without changing accepted results. Do not cache complete policies until C1 is repaired.

See bracket_probe.json S028 and its adjacent executable source.

3. Mathematical review of the active path

The maintained household state is liquid wealth, inherited tenure/owner size, age, income, lifetime parity, and the number of children at home. Location remains a singleton axis. Decisions run from fertility attempt and birth realization to current tenure/housing choice, saving, income and child transitions, mortality, and next-period entry. Several distributions are needed because housing is observed after tenure choice while wealth targets use the relevant beginning-asset convention.

Objects that match their coded specification

Preferences and housing: with m dependent children, the floor is

$$\bar{h}(m) = \bar{h}_J \mathbf{1}\{m > 0\} + \bar{h}_n m.$$

The fixed consumption share and equivalence scale enter

$$u(c, s; m) = \frac{e(m)^{\sigma-1}\left[c^{\alpha} s^{1-\alpha}\right]^{1-\sigma}}{1-\sigma} + \psi m, \qquad e(m) = \left(\frac{2 + 0.7m}{2}\right)^{0.7}.$$

Here $s = h - \bar{h}(m)$ for renters and $s = \chi[H - \bar{h}(m)]$ for owners. The owner premium changes utility services; physical demand remains H. Preference construction S005 and owner kernel S029 agree on that distinction. Conditional on saving, the renter allocation follows the Cobb-Douglas budget shares and switches to the capped branch when needed. The separate reporting floors discussed below remain a qualification.

Purchase and collateral: a newly purchased house costing PH requires $(1 - \phi)PH$ in cash, and the post-transaction liquid position subtracts the full purchase price. Sale proceeds are net of the sale wedge. The cash test and collateral floor agree algebraically, including sale proceeds for movers. The independent threshold check verifies both sides of the cash boundary. Existing secured debt is separated from the unsecured rollover allowance; it is not counted twice in the inspected owner floor.

Sequential fertility: first-birth and continuation choices compare waiting with attempting, account for conception probability, and charge the fixed first-birth utility cost only upon success. The forward operator snapshots higher-parity at-risk pools before incoming births arrive. The direct test starting with one childless and one one-child household creates one first birth and one second birth, with no household reaching parity three in that period.

Income and child transitions: the Bellman expectation and distribution transition use opposite orientations of the same income and child-transition matrices. Under the maintained child-count process,

$$m' \mid m \sim \text{Binomial}(m, 7/9).$$

A deterministic randomized test checks the adjoint identity $\langle Tg, V\rangle = \langle g, T^*V\rangle$ with nontrivial income and maturation, identity tenure/saving maps, and all ten valid family states. Both the value-pairing discrepancy and mass discrepancy are exactly zero in that test. This does not test optimal saving or every transaction boundary.

Mortality and renewal: age survival weights incumbent advancement and the corresponding death/bequest branch. Terminal households exit. A four-slot birth queue, consumed when building the next dated state, sends an isolated date-0 birth impulse to the entrant state twenty years later. The factor dividing births by 2.1 is explicitly a household-renewal normalization. Its arithmetic is internally consistent; its empirical household-formation interpretation remains outstanding.

Target calculation: the maintained objective uses the ordered target system and fixed weights; nonfinite dated moments fail before scoring. The active first-birth rooms row uses a separate 2019 treated/control branch advanced to 2023 with dated policies, and permits continuation fertility only in the treated branch. The code distinguishes this target from the older same-policy diagnostic. The control remains childless by construction. This is the declared model measurement, not proof of equivalence to the empirical estimator.

Known limitations that this review does not turn into new bugs

- **Saving maximization:** the active local golden-section search does not establish global optimality over a piecewise-linear continuation value. Previous exact oracle work found rare failures, and the lead reports two more very small occupied exposures in nonwinning morning candidates. The overnight global-saving comparison and conditional grid refinement support the main tested aggregate contrast, but do not certify every candidate or derivative. The active Markov path sets `has_prev=0`; the old "previous saving ±2" heuristic is not the explanation for these active results.
- **Reported consumption/housing floors:** the kernel can replace very small optimal quantities with reporting floors without re-solving the budget. This known inconsistency had negligible occupied aggregate exposure at the selected state in the overnight audit. A wealth-value feasibility gate is not a substitute for explicit household budget checks.
- **Age and family-group alignment:** rounding requested ages to model nodes is not interval-overlap measurement. The active family ownership gap also compares dependent-child parents with never-parents, whereas the empirical recent-parent and no-resident-child groups differ. These are substantive target-mapping limitations, already identified in the prior audit; simplification cannot silently resolve them.
- **Initial versus dated supply:** the initialization still sets elasticity 1.75 in the chain, while the externally fixed 0.63 is applied to the dated supply rule after the old equilibrium is built. This is visible in the stored supply normalization. Whether this two-stage contract is the intended economic specification requires an explicit decision; changing it would require re-estimation, not an unnoticed cleanup.
- **Written budget timing:** the code applies the gross bond return to post-transaction liquid wealth, $R(b + T_H)$, while older draft text used $Rb + T_H$. Beginning-of-period trading gives the code an interpretation, but the two formulas are not interchangeable. The old implementation note is also inaccurate when it says purchase principal is not subtracted.
- **Economic closure:** warm-glow bequests do not implement inheritance receipts for descendants; historical age raking imposes household formation; pensions and the property-tax rebate are distinct fiscal accounts; temporary-equilibrium paths are not perfect-foresight welfare transi-

tions. The separate person-cohort/perfect-foresight experiments have not been certified by this review.

4. Where the bloat actually is

Counts below describe physical Python lines, including comments and blanks. They are organization evidence, not measures of computational cost.

Layer	Evidence	Interpretation
Current sequential package	47 top-level Python files; 20,123 lines	Includes core, inherited infrastructure, experiments, collectors and reports
Sequential modules imported at initialization	17 modules; 14,899 lines	Import reachability, not evidence that every function executes
Other top-level sequential files	30 files; 5,224 lines	Mostly separate experiments and reports; not demonstrated dead code
All project modules imported during calibration initialization	33 modules; 36,508 lines	Includes the old one-shot package before `calendar.model` is replaced
Core sequential solver	7,428 lines	Bellman, several equilibrium routes, distribution, statistics, policy accounting and reporting are mixed
Clearly non-Markov route functions within that solver	10 functions; 1,789 lines	Not dispatched for the maintained Markov-income route; useful historical/reference code, not universally unused
Byte-identical modules shared with the older package	4 files; 1,774 lines	`local_panel`, `production_profile`, `target_system`, and `state_layout` are copied verbatim

The initialization inventory covers imports triggered by calibration setup; later model setup can load additional small profile modules. It is not an execution trace or a complete dynamic dependency graph. `state_layout.py` supplies a named axis contract but is not used by the active solver; current state manipulation instead relies on positional axes throughout.

Several long functions have distinct responsibilities that can be separated without changing economics: calibration `main` spans 1,036 lines, the Markov forward function 686, the general statistics function 635, and the active Markov Bellman function 464. The stationary forward implementation and calendar cohort advancement separately encode related fertility and transition logic. A bug fix must therefore be reconciled across multiple sites. C1 is a concrete example: stationary cached policies were repaired while the calendar policy object retained the old side channel.

Not every repeated pathway should immediately be deleted. Reference kernels and old model variants provide useful equivalence checks. The practical goal is a narrow active interface with explicit historical routes, rather than concealing all code in one shared abstraction or copying another complete solver fork.

5. Performance findings and realistic opportunities

Measured cost first

The selected production task's saved receipt reports **454.76 seconds total**, including **232.97 seconds in five old-steady-state normalization evaluations** and **211.81 seconds in the dated candidate scenario with 50 Bellman solves**. These are dated Torch receipts, not a new benchmark on the laptop. The initial normalization accounts for about 51% of that case's wall time; the dated scenario about 47%. Their total leaves roughly ten seconds for other work. The scenario time divided by its solve count is not a pure Bellman kernel timing because distribution and target measurement also occur in that scenario.

Thus the first performance question is how to reduce complete solves, and then how to reduce work inside each solve. The 36,508 imported lines do not imply that removing imports saves a similar fraction of runtime. Imports should be cleaned up principally for clarity and reliable configuration.

Concrete opportunities, ranked

Change	Evidence and likely payoff	What must be verified
Stop repeated boundary evaluations	Proven 19-call/two-price failure example; large saving on that failure path, none claimed on ordinary accepted paths	Same explicit failure at same bounds; unchanged successful solutions
Reuse the old normalized equilibrium for candidates differing only in the dated preference change	The old equilibrium depends on structural parameters and old normalization, not the terminal preference change; it consumes about half of the observed case	Cache key includes every relevant primitive, source, grid and normalization tolerance; exact history reproduction
Use safeguarded interpolation/Brent logic for dated market roots, with exact-price caching	The stationary solver already has a direct scalar route; the dated solver bisects and performs many full solves	Same market gate, no false convergence across jumps, C1 fixed first; benchmark solve counts and complete fit
Compile or fuse the active Markov distribution loops	Active forward and cohort routines repeatedly scatter mass inside Python loops; Numba scatter primitives exist, but the all-in-one compiled forward kernel is on the non-Markov route	Operator adjoint, nonnegativity, mass, child/entrant flows and full dated-state equality
Skip structurally invalid family columns	Only ten of the sixteen parity/at-home-count pairs are reachable with readiness off; saved invalid-state mass is exactly zero	Reachability proof including births/maturation, owner choices and bequests; preserve a diagnostic adapter and all valid-state policies
Specialize singleton location and renter-only housing storage	Maintained location count is one; owner physical housing is fixed by its rung	Preserve the location utility constant and dead-state behavior; no change to wealth/tenure timing
Reduce repeated full-statistics and duplicated arrays	The Markov statistics path collapses to generic statistics then recomputes nonlinear moments; old-state normalization retains all solved objects	Keep necessary target estimands explicit; compare every target and required diagnostic, not just the loss

The saved policy arrays alone occupy **201.23 MB**, excluding maps, distributions and continuation probabilities; the latter add **23.50 MB**. The dense tenure-probability array uses **70.50 MB** even in float32. Each ordinary seven-dimensional float64 state array uses **23.50 MB**. This explains why retaining many price or normalization candidates is costly. `solve_old_steady_state` caches the entire solution and parameters for every tried intercept, even though its root search needs far fewer full payloads simultaneously. Keeping scalar records for all trials and full arrays only for necessary candidates can reduce memory without changing a solve.

There are **six unreachable family columns out of sixteen**. Omitting them would remove 37.5% of the corresponding storage and household-column work, not necessarily 37.5% of total runtime. Likewise, eliminating a stored singleton-location probability array saves 23.50 MB for this grid, but its runtime effect needs measurement. Sparse or compressed storage is an interface change and should follow simpler profiling and cache repairs.

I do **not** recommend decreasing the wealth grid, income-state count, target count, tolerances, or global-search accuracy in the name of speed. Those alter the research object or its approximation. Nor is adding more within-solve threads an established gain: the compiled block loops parallelize over only sixteen family columns, while the current cluster workflow already parallelizes independent cases.

6. Coverage and validation record

Component	Reviewed or executed here	Remaining uncertainty
Source and active architecture	Production/checkout manifest comparison, import inventory, AST function inventory, copied-file comparison	Full execution coverage of every conditional route was not collected
Utility, owner/renter branches, down payment	Read active formulas and kernels; direct threshold test on both sides	Compiled optimal-saving equivalence and every budget boundary were not rerun
Bellman/forward income and maturation	Direct adjoint and mass test; inspected survival and family indexing	Full optimal-policy adjoint across every purchase boundary deferred
Sequential births and renewal queue	Direct one-birth test and queue impulse; inspected top-code flow conversion	Empirical household-formation interpretation remains unresolved
Stationary and dated market solve	Traced active dispatch, cache restoration, retry and bracketing; two targeted defect probes	No new GE solve or cycle-wide profiler
Dated SMM and first-birth housing measurement	Read old normalization, dated branches, timing and cross-section extraction; six existing unit functions directly invoked	Raw empirical estimators and target redesign were outside this code pass
Mortality, eligibility and fiscal primitives	Nine additional existing tests directly invoked for survival, maturation, eligibility, transfers and grant arithmetic	Full rebated transition and consolidated fiscal closure not recertified
Production policy exposure	Read and hash-checked the five saved 11-date fallback columns	Other historical/experimental policy packets not exhaustively scanned
Diagnostics and current runner	Checked entry-point routing, moment labels and array conventions	No graph regeneration; overnight packet inspection remains prior evidence
Experimental and legacy code	Mapped imports and isolated inactive routes	E6 readiness, PTI, owner LTV taper, spatial multi-market, person-cohort and perfect-foresight variants not fully audited

Executed checks: fifteen existing test functions passed by direct invocation against the frozen runtime, plus the independent down-payment, forward-adjoint, one-birth and queue tests. Three constructed probes establish the policy side channel, wrong verbose fertility units, and redundant boundary evaluations. Saved-state checks confirm the 120-node shape, 15 income states, ten valid family states, memory counts, and no production fallback dates.

Environment limits: the project pytest command exited with native code 139 and no test output, including a retry with JIT disabled and a retry with automatic plugin loading disabled. This was not classified as a model-code failure or a passing test suite. Direct test-function execution succeeded using Python 3.10.10, NumPy 1.24.3 and disabled Numba JIT. Python 3.9.6/NumPy 2.0.2 was used for compatible saved-checkpoint reads and the no-solve API probes. Python 3.9 cannot run the full parameter setup because it uses runtime union-type checks. The bundled Python 3.12 runtime lacks matplotlib, which the diagnostic module imports at initialization. No environment was modified or package installed. No compiled-kernel or full production test suite is claimed in this pass.

7. Staged plan

1. **Small correctness and navigation patch, roughly half to one working day:** C1-C4, one maintained entry point, a compact model flow map, and a strict check that sequential calendar operators are installed. Keep the current public shape temporarily; add focused stale-policy/retry tests. Reproduce the selected calibration and one supply/credit initial-state comparison before merging any numerical-interface change.
2. **Measured speed pass, roughly one to two days:** one exact-loop Torch profile with separate Bellman, current-choice realization, advancement, statistics, and serialization timings. Benchmark root-solve improvements and safe reuse of the old normalization. Record solve counts as well as wall time and peak memory. Adopt only gains that preserve gates and complete fit/parameter receipts.
3. **Small structural extraction, several days:** separate active household solving, transition operators, measurements, and reporting from legacy dispatch. Move or label experimental entry points; replace environment/import mutation with an explicit model configuration. Keep wrappers so saved outputs and regeneration commands remain usable.
4. **Optional larger optimization after that evidence:** compressed family-state arrays, fused Markov advancement, and more global saving optimization. These need valid-state/transaction-boundary tests, strict full-history repetition, the stable seventeen-figure packet, and renewed policy checks. Do not promise a speed multiplier before profiling.

The first two stages can materially improve reliability and working speed while preserving the model. A full rewrite would create a large verification burden precisely when target measurement and economic specification still need decisions.

Evidence package and reproduction

All read-only evidence and executable probes are in the architecture audit packet S030. Its JSON and CSV files record source identities, detailed footprint counts, checks, and saved-path exposure. The report is the only source file written by this reviewer; model files, other agents' edits, and the protected manuscript were preserved.

Lead verification

The lead independently repeated all fifteen selected test functions and the four arithmetic invariants, reproduced the policy-state defect, and inspected all fifty-five saved production retry flags. The formulas for preferences, owner services, the dated supply override and the legacy fertility log were checked against the frozen source. The review's distinction between a demonstrated API defect and an affected production result is retained. No solver repair or production promotion is part of this document.

Repair addendum: the first practical cleanup

The author's follow-up asked for obvious improvements with correctness first. This patch changes the calendar policy interface and removes redundant work. It does not change household preferences, constraints, the saving optimizer, target values, weights, parameter bounds, grids or acceptance tolerances.

C1 repaired: saved household decisions carry all birth probabilities. All three policy factories now retain their own copy of later-birth probabilities. The calendar fertility operator, dated first-birth housing branch and diagnostic readers use that copy explicitly. A new solve can no longer change the birth probabilities used when an earlier policy is reused. Old incomplete sequential policy objects fail clearly and must be rebuilt from their matching solution or solved again; no automatic imputation from mutable parameter state is allowed. A policy supplied at a different price is also rejected. Callers still own invalidation when preferences or policy primitives change; the independent follow-up traced the maintained callers without finding another blocker.

C4 repaired for bracket expansion. When expansion reaches a price bound, the search evaluates that bound once and then reports failure if no root is bracketed. The isolated failure example falls from nineteen evaluations at two prices to two evaluations, with the same explicit failure. This is a saving on that failure path, not a measured overall speed multiplier. Initial prices outside the declared bounds fail before solving. The redundant final bisection-midpoint evaluation remains a small separate cleanup.

C2 clarified: one short reading map. The sequential package README points to canonical status, the household solver, dated target measurement, population operators and the existing diagnostic PDF. The June runner and implementation record are explicitly historical. Inspecting the saved result needs no new solve. Historical executable routes are preserved for reproducibility.

Verification. Nine focused regression checks and eighteen existing accounting checks pass locally and on Torch. Five full compiled Bellman replays exactly reproduce every saved policy array, continuation probability, distribution, birth total and market residual: the production 2023 state, the morning review candidate, and the baseline, supply and credit diagnostic impacts. Reuse and serialization/reload remain exact after deliberately corrupting the solver's scratch probabilities. Every replay writes the unchanged seventeen-graph packet. The complete five-date historical replay also reproduces all twelve target rows, all parameter and bound rows, and 253 numeric history entries exactly. Complete tables and receipts are in the repair evidence folder.

Still outstanding. C3, the misleading verbose fertility formula, is unchanged. Separating old and experimental routes, splitting large functions, consolidating duplicate modules and reducing memory use need a separate profiled pass. Keeping a copied continuation array adds about 23.5 MB per retained production policy; that cost buys correct reuse and should precede more aggressive caching. The interpretation of the initialization elasticity and the economic issues in the review remain open. No calibration or policy result is promoted by this patch.

The reproducible repair packet is `output/model/e5f_policy_cleanup_verification_20260905a/README.md`. Its final scientific bundle is `33167d84113e2bd38d9ee48dcd9ab0403790348610d998d4032fb8c1797ad3e3`. Original source and target fingerprints remain intact in their frozen packets; production commands must continue to verify their own declared source contract.

Source index

References reproduce the paths and line numbers used during the audit. Local source files may subsequently change. Run folders and saved receipts identify the reviewed artifacts; web links identify the cited primary sources.

S001. CALIBRATION_STATUS.md

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/CALIBRATION_STATUS.md

S002. intergen_eqscale_seq_optimized/solver.py

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/solver.py

S003. full audit source

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/docs/model/e5f_independent_quantitative_audit.md

S004. calibration main

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_e5f_transition_calibration.py:1627

S005. precompute_shared

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/solver.py:2213

S006. solve_bellman_full_markov_income

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/solver.py:2344

S007. renter and owner kernels

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/kernels.py:761

S008. advance_cohort_one_period_markov_income

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/solver.py:5179

S009. begin_dated_first_birth_housing_branch

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_e5f_transition_calibration.py:1006

S010. run_birth_vintage_scenario

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_e5f_open_population_transition.py:1019

S011. apply_policy

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_e5f_post2023_policy_mechanisms.py:138

S012. PolicyBundle

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_dynamic_population_transition.py:67

S013. solve_policy

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_dynamic_population_transition.py:296

S014. evaluate_period

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_dynamic_population_transition.py:421

S015. sequential fertility

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_e5f_open_population_transition.py:688

S016. the existing stationary repair

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/solver.py:1552

S017. tighter-tolerance retry

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_e5f_open_population_transition.py:1172

S018. post-2023 evaluate_state retry

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_e5f_post2023_no_policy_continuations.py:905

S019. the JSON receipt

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/output/model/e5f_code_architecture_audit_20260905a/side_channel_probe.json

S020. the executable reproduction

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/output/model/e5f_code_architecture_audit_20260905a/side_channel_probe.py

S021. package README

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/README.md:8

S022. runner settings

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/run_intergen_model.py:28

S023. runner dispatch

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/run_intergen_model.py:70

S024. run_model_cp_dt output

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/solver.py:1137

S025. fixed-price output

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/solver.py:1547

S026. authoritative extraction

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/calibration.py:1297

S027. calendar market bracketing

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/tools/run_dynamic_population_transition.py:509

S028. bracket_probe.json

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/output/model/e5f_code_architecture_audit_20260905a/bracket_probe.json

S029. owner kernel

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/code/model/intergen_eqscale_seq_optimized/kernels.py:957

S030. the architecture audit packet

/Users/tommasodesanto/Desktop/Projects/Fertility/Fertility_Spring26/output/model/e5f_code_architecture_audit_20260905a/README.md